CASB & Cloud App Control with Netskope

Implementation Solution Guide

Version 1.0.0 · 2026-06-29

Contents

Executive Summary — CASB & Cloud App Control with Netskope 4

What you get . . . 4

What this delivers . . . 4

The standard deployment model . . . 4

Outcome . . . 4

1. Solution Overview — CASB & Cloud App Control with Netskope 5

What CASB is . . . 5

Why this matters . . . 5

How Netskope CASB works . . . 5

Cloud app discovery and risk scoring (sanctioned vs shadow IT) . . . 5

The control levers . . . 6

Govern without blocking . . . 6

Limitations (be honest with buyers) . . . 6

The standard deployment model . . . 6

2. Requirements Guide — CASB & Cloud App Control with Netskope 6

2.1 Netskope tenant with the CASB module licensed . . . 7

2.2 Traffic steering / SWG already in place . . . 7

2.3 SSL/TLS inspection enabled for the in-scope apps . . . 7

2.4 Microsoft Entra ID identity for users and groups . . . 7

2.5 API connections to sanctioned apps (for API CASB) . . . 7

2.6 A sanctioned-app inventory (the starting register) . . . 7

2.7 A pilot group and a rollback path . . . 8

Checklist . . . 8

3. Implementation Guide — CASB & Cloud App Control with Netskope 8

Step 1 — Build the scope groups (Assess) . . . 8

Step 2 — Discover and risk-score cloud apps (Assess) . . . 8

Step 3 — Decide sanctioned vs unsanctioned (Configure) . . . 9

Step 4 — Configure inline activity controls (Configure) . . . 9

Step 5 — Configure API controls for sanctioned apps (Configure) . . . 9

Step 6 — Apply on the pilot (Pilot) . . . 9

Step 7 — Validate the pilot (Validate · gate) . . . 9

Step 8 — Remediate policy gaps (Validate) . . . 10

Step 9 — Expand to production (Expand) . . . 10

5. Testing Guide — CASB & Cloud App Control with Netskope 10

Test 1 — Discovery populated and scored . . . 10

Test 2 — Sanctioned app allowed . . . 11
Test 3 — Restrict / coach on an unsanctioned app . . . 11
Test 4 — Inline activity control fires . . . 11
Test 5 — Data-movement control catches sensitive content . . . 11
Test 6 — API connection governs data at rest . . . 11
Test 7 — Pilot scoping correct . . . 11
Test 8 — Usage & risk report . . . 11
Pass/fail gate . . . 11
6. Troubleshooting Guide — CASB & Cloud App Control with Netskope 12
A. Controls not firing / app not seen by policy . . . 12
B. Over-blocking / business impact . . . 12
C. Discovery / risk register gaps . . . 12
D. API CASB issues . . . 13
E. Identity / scoping . . . 13
F. Diagnostics quick reference . . . 13
G. When to escalate . . . 13
7. Best Practices — CASB & Cloud App Control with Netskope 13
8. Deliverables Summary — CASB & Cloud App Control with Netskope 14
Documentation . . . 14
PowerShell scripts . . . 14
Named artifacts (reused verbatim across the package) . . . 15
What the buyer receives . . . 15
Outcome . . . 15
Appendix A — PowerShell Scripts 16

Executive Summary — CASB & Cloud App Control with Netskope

Your people use hundreds of cloud apps, most of which IT never approved, and corporate data flows into them unseen. Blocking everything kills productivity; allowing everything is a data-loss and compliance liability. A Cloud Access Security Broker (CASB) gives you the third option: see every cloud app in use, score its risk, and apply graduated controls so cloud usage is governed without blocking the business.

This is a complete, deploy-it-the-repeatable-way package for delivering CASB and cloud app control with Netskope, written for an administrator with no prior CASB experience. Netskope console specifics change over time and are deferred to the official Netskope documentation; this package operates at the correct architectural and procedural level and names every artifact so the work is repeatable.

What you get

- A plain-language Solution Overview of how Netskope CASB works (inline and API), cloud app discovery

and risk scoring (sanctioned vs shadow IT), the control levers (allow, restrict, coach, block, activity and data-movement controls), and the govern-without-blocking approach.

- A Requirements Guide with verify / implement-if-missing for every prerequisite.
- An Implementation Guide of precise, ordered steps: discover and risk-score apps, define

sanctioned/unsanctioned policy, apply inline activity controls and API controls, validate, pilot, and expand.

- Five PowerShell scripts for cloud-app discovery readiness, control-policy validation, a cloud-app

usage and risk report, safe remediation of policy gaps, and a CASB-scope group builder.

- Testing and Troubleshooting guides with explicit pass/fail gates, plus field-proven Best Practices.

What this delivers

A governed cloud-app estate: shadow IT discovered and risk-scored against an app risk register, a clear sanctioned vs unsanctioned policy, inline activity and data-movement controls on risky activities, and API connections governing data already resident in sanctioned apps — all proven on a pilot group before fleet rollout, with a documented rollback.

The standard deployment model

Assess -> Configure -> Pilot -> Validate -> Expand.

Outcome

Every cloud app visible and governed — shadow IT discovered and risk-scored, with inline and API controls that protect data without blocking the business — proven on a pilot before you scale.

1. Solution Overview — CASB & Cloud App Control with Netskope

What CASB is

A Cloud Access Security Broker (CASB) sits between your users and the cloud apps they use and gives you three things you cannot get from a firewall alone: visibility into which cloud apps are used, a risk score for each app, and granular control over what users can do inside those apps. Netskope delivers CASB in two complementary modes — inline (proxying live traffic) and API (connecting directly to the management plane of a sanctioned app) — so you can govern both data in motion and data already at rest in the cloud.

Why this matters

Most cloud usage is invisible to IT. Employees adopt apps on their own (shadow IT), and corporate data flows into them with no policy, no logging, and no way to retract it. The traditional response — block the category at the firewall — fails because it is all-or-nothing: it blocks the sanctioned app alongside the risky one, drives users to workarounds, and still misses data movement inside an allowed app. CASB replaces that blunt instrument with graduated control: allow the safe, restrict the risky activity, coach the user toward the sanctioned alternative, and block only what truly warrants it. The goal is to govern cloud usage without blocking the business.

How Netskope CASB works

1. Inline CASB steers live web/cloud traffic through Netskope (via the SWG/steering already in place), so policy can see and act on the activity in real time — uploads, downloads, posts, shares.

1. API CASB connects out-of-band to a sanctioned app's API (for example a sanctioned file-storage or collaboration suite) to inspect and govern data already resident there — exposed shares, risky files, external collaborators — without being in the traffic path.

1. Cloud app discovery analyzes traffic and logs to surface every app in use, sanctioned or not, and assigns each a risk score (the Cloud Confidence Index / app risk rating).

1. Policy applies control levers per app and per activity, scoped to identity (users and groups from Entra ID).

1. Activity is logged and reported, feeding the app risk register and the usage/risk reports.

Cloud app discovery and risk scoring (sanctioned vs shadow IT)

• Discovery distinguishes sanctioned apps (IT-approved, often API-connected) from unsanctioned / shadow-IT apps (in use but not approved).

• Each discovered app carries a risk score derived from its security, compliance, and data-handling attributes. The score drives the policy decision: sanction it, restrict it, coach away from it, or block it.

• The named output is the app risk register — All cloud apps – Discovery stage — a living inventory

of discovered apps, their risk score, and the disposition decision.

The control levers

- Allow — permit the app/activity (typically the sanctioned app).
- Restrict — permit the app but constrain specific activities (for example allow download, restrict upload of sensitive data).
- Coach — let the user proceed but present guidance steering them to the sanctioned alternative, or require a justification.
- Block — deny the app or activity outright (reserved for genuinely unacceptable risk).
- Activity controls — act on what the user does (upload, download, share, post, edit, login).
- Data-movement controls — act on the content being moved (used with DLP-style profiles to catch sensitive data leaving for an unsanctioned destination).

Govern without blocking

The operating principle is least-disruption: prefer restrict and coach over block, scope every new control to a pilot group first, and reserve hard blocks for the highest-risk apps on the register. This keeps users productive on sanctioned apps while closing the data-loss path through shadow IT.

Limitations (be honest with buyers)

- Inline control requires traffic steering / SWG already in place (a category prerequisite).
- API control requires a supported, sanctioned app and an authorized API connection to it; coverage depends on what each app's API exposes.
- Discovery and risk scores are a decision aid, not a verdict — disposition is a human policy choice.
- Exact Netskope console screens, menu paths, and tenant options change over time and are deferred to the official Netskope documentation and your tenant.

The standard deployment model

> *Discover and risk-score apps -> define sanctioned/unsanctioned policy -> apply inline activity and API controls -> validate -> pilot, validate, expand. Model: Assess -> Configure -> Pilot -> Validate -> Expand.*

2. Requirements Guide — CASB & Cloud App Control with Netskope

Verify every requirement before configuring. Each entry: Description · Why · Verify · Implement-if-missing. Category-wide prerequisites are in `global-requirements-network.md`.

2.1 Netskope tenant with the CASB module licensed

Description: An active Netskope tenant with the CASB module (inline and API) licensed and enabled. Why: CASB is the control plane for discovery, risk scoring, and the allow/restrict/coach/block levers. Verify: Sign in to your Netskope tenant as an administrator and confirm the CASB capability is present. Implement: Engage your Netskope account team / reseller to add the CASB module to the tenant.

2.2 Traffic steering / SWG already in place

Description: Live web/cloud traffic is already steered through Netskope (client or steering config). Why: Inline CASB and activity controls only see and act on traffic that flows through Netskope. Verify: Confirm steering is active for the target population; run `NSCASB-Validation.ps1` for the steering-readiness signals. Implement: Deploy/assign the Netskope client and steering configuration (the category steering/SWG prerequisite) before enabling inline control.

2.3 SSL/TLS inspection enabled for the in-scope apps

Description: Decryption is enabled so Netskope can inspect activity inside HTTPS cloud-app traffic. Why: Without inspection, inline activity and data-movement controls cannot read the activity to act on it. Why-not-everywhere: Some apps/categories are exempted by policy; confirm the in-scope apps are inspected. Verify: Confirm an inspection policy covers the target cloud-app categories; client decryption certificate is trusted on endpoints. Implement: Enable SSL inspection for the in-scope categories and deploy the decryption certificate (via Intune — category prerequisite G3).

2.4 Microsoft Entra ID identity for users and groups

Description: Users and groups resolve from Microsoft Entra ID for identity-based policy scoping. Why: Every control is scoped to identity (the pilot group, then broader groups), not to IP. Verify: `https://entra.microsoft.com` -> Groups; run `NSCASB-Discovery.ps1` to confirm Graph connectivity and group readiness. Implement: Ensure Entra ID is the identity source and groups exist (sync Active Directory if hybrid — category prerequisite G5).

2.5 API connections to sanctioned apps (for API CASB)

Description: Authorized API connections from Netskope to each sanctioned app you intend to govern at rest. Why: API CASB governs data already resident in a sanctioned app (exposed shares, risky files) out-of-band. Verify: In the Netskope tenant, confirm an API connection (instance) exists and is healthy for each sanctioned app. Implement: Establish the API connection per Netskope's documented connector flow for that specific app; authorize least-privilege access.

2.6 A sanctioned-app inventory (the starting register)

Description: A first-draft list of which cloud apps are officially sanctioned vs not. Why: Discovery refines it, but you need a baseline to make sanctioned/unsanctioned policy decisions against. Verify: Run `NSCASB-Discovery.ps1` to enumerate Entra enterprise applications as a cloud-app inventory signal; reconcile with Netskope discovery. Implement: Draft the sanctioned list with the business; record each app's disposition in the app risk register.

2.7 A pilot group and a rollback path

Description: An Entra security group of pilot users for staged control rollout, plus a documented rollback. Why: Every control change is piloted and must be reversible (inline enforcement affects live traffic). Verify: Entra -> Groups; confirm the pilot group exists (build it with `NSCASB-Set-ScopeTargeting.ps1`). Implement: Create the pilot group; write the rollback (scope-out / set control to monitor-only / disable policy) — category prerequisites G6 and G7.

Checklist

- Netskope tenant with CASB module licensed
- Traffic steering / SWG in place — `NSCASB-Validation.ps1`
- SSL/TLS inspection enabled for in-scope apps
- Entra ID identity + groups — `NSCASB-Discovery.ps1`
- API connections to sanctioned apps (for API CASB)
- Sanctioned-app inventory drafted into the app risk register
- Pilot group created and rollback documented — `NSCASB-Set-ScopeTargeting.ps1`

3. Implementation Guide — CASB & Cloud App Control with Netskope

Follow these steps in order. Configuration is performed in your Netskope tenant; exact console screens and menu paths change over time and are deferred to the official Netskope documentation — follow the named step and confirm the precise control in your console. Identity and scoping use Microsoft Entra ID.

> *Model: Discover and risk-score apps, decide sanctioned vs unsanctioned, apply inline activity controls and API controls for sanctioned apps, then pilot, validate, and expand. Every named artifact below is reused verbatim in Testing, Troubleshooting, and the scripts.*

Step 1 — Build the scope groups (Assess)

1. Build the pilot scope group `CASB Pilot – Pilot users – Pilot` with `NSCASB-Set-ScopeTargeting.ps1`

(a few representative users across roles).

1. Build the broad scope group `CASB Broad – All users – Expand` for the eventual fleet rollout.
2. Confirm both groups resolve in Entra -> Groups.

Step 2 — Discover and risk-score cloud apps (Assess)

1. In Netskope, run cloud app discovery over recent traffic/logs to surface every app in use.
2. Collect the corroborating inventory signal: run `NSCASB-Discovery.ps1` to enumerate Entra

enterprise applications (a sanctioned-app signal) and confirm Graph + group readiness.

1. Record the result as the app risk register — All cloud apps – Discovery stage: each app with its risk score and a draft disposition (sanction / restrict / coach / block).

Step 3 — Decide sanctioned vs unsanctioned (Configure)

1. With the business, finalize each app's disposition in the app risk register.
2. Name the two policy artifacts you will build:

• Sanctioned apps policy – In-scope apps – Configure (allow + governed controls).

• Unsanctioned apps policy – Shadow IT – Configure (restrict / coach / block per risk score).

Step 4 — Configure inline activity controls (Configure)

1. Build the activity controls — Inline – Configure profile: define which activities are controlled (upload, download, share, post, login) and the lever per activity (allow / restrict / coach / block).

1. For sensitive-data movement, attach a data-movement control (DLP-style profile) so sensitive content moving to an unsanctioned destination is restricted or blocked.

1. Apply the levers per the app risk register: coach users from a risky app toward the sanctioned alternative; restrict risky activities on tolerated apps; reserve block for the highest-risk apps.

1. Scope every inline policy to `CASB Pilot – Pilot users – Pilot` first. Confirm with `NSCASB-Validation.ps1`.

Step 5 — Configure API controls for sanctioned apps (Configure)

1. Confirm the API connections — Sanctioned apps – Configure exist and are healthy for each API-governed sanctioned app (Requirement 2.5).

1. Build the at-rest governance policy on each connection: detect exposed/public shares, external collaborators, and risky files; choose the response (alert, restrict sharing, quarantine) per the register.

1. Begin in monitor/alert mode so you can see findings before any automated response acts.

Step 6 — Apply on the pilot (Pilot)

1. Confirm the pilot scope group is the target of the inline policies and (where applicable) the API findings are reviewed for pilot users.

1. From a pilot endpoint, exercise representative activities (upload to sanctioned vs unsanctioned, share externally, download).

1. Confirm controls fire as expected; run `NSCASB-Validation.ps1`.

Step 7 — Validate the pilot (Validate · gate)

1. Run `NSCASB-Validation.ps1` -> confirms steering/inspection readiness and that the named policies

are scoped to the pilot group.

1. Run `NSCASB-Report.ps1` -> produces the cloud-app usage and risk report and confirms the app risk

register is populated.

1. Complete the Testing Guide. Pilot gate: do not expand until the pilot pass gate is met.

Step 8 — Remediate policy gaps (Validate)

1. If validation finds gaps (a sanctioned app with no policy, an unsanctioned high-risk app with no

control, an unhealthy API connection, missing pilot scoping), run `NSCASB-Remediation.ps1`.

1. `NSCASB-Remediation.ps1` defaults to `-WhatIf` (reports the fix it would make); re-run with

`-Execute` only after review.

Step 9 — Expand to production (Expand)

1. After the pilot gate passes, re-scope the named policies from `CASB Pilot – Pilot users – Pilot`

to `CASB Broad – All users – Expand` in stages.

1. Promote API at-rest policies from monitor mode to active response only after pilot validation.

2. Re-validate each cycle with `NSCASB-Report.ps1`; keep the app risk register current as discovery

surfaces new apps.

> *Rollback note: to reverse a control, scope the policy back to monitor-only or remove the pilot/broad group from its assignment, and (for API) revert the connection's response to alert-only. Inline enforcement affects live traffic — keep the rollback ready before enabling block.*

Order of operations

```
Scope groups -> Discover & risk-score -> Sanctioned/unsanctioned decision ->
Inline activity controls -> API controls -> Apply on pilot -> Validate (gate)
-> Remediate gaps -> Expand
```

5. Testing Guide — CASB & Cloud App Control with Netskope

Run after the pilot policies apply (Implementation Step 7) and after each expansion stage. Use the named artifacts from the Implementation Guide.

Test 1 — Discovery populated and scored

Do: Confirm the app risk register — All cloud apps – Discovery stage is populated; run `NSCASB-Report.ps1`. Expected: Discovered apps are listed with a risk score and a disposition;

sanctioned vs unsanctioned is distinguished.

Test 2 — Sanctioned app allowed

Do: From a pilot endpoint, use a sanctioned app for normal work (login, view, edit). Expected: The sanctioned app works normally; the Sanctioned apps policy – In-scope apps – Configure allows it.

Test 3 — Restrict / coach on an unsanctioned app

Do: From a pilot endpoint, attempt the controlled activity (for example upload) on an unsanctioned app. Expected: The Unsanctioned apps policy – Shadow IT – Configure restricts, coaches, or blocks per the register — not all-or-nothing.

Test 4 — Inline activity control fires

Do: Exercise a controlled activity (upload, external share, download) covered by the activity controls — Inline – Configure profile. Expected: The lever applies (allow / restrict / coach / block) on the activity, scoped to the pilot group; confirm with `NSCASB-Validation.ps1`.

Test 5 — Data-movement control catches sensitive content

Do: Attempt to move sensitive test content to an unsanctioned destination. Expected: The data-movement control restricts or blocks the transfer per policy.

Test 6 — API connection governs data at rest

Do: For a sanctioned API-governed app, create a test exposed share / external collaborator. Expected: The API connections — Sanctioned apps – Configure surface the finding (and apply the chosen response once promoted from monitor mode).

Test 7 — Pilot scoping correct

Do: Run `NSCASB-Validation.ps1`. Expected: The named policies are scoped to `CASB Pilot – Pilot users – Pilot` (not the whole fleet) before expansion.

Test 8 — Usage & risk report

Do: Run `NSCASB-Report.ps1`. Expected: A clear cloud-app usage and risk report — sanctioned vs shadow IT, per-app risk score, and policy coverage gaps.

Pass/fail gate

Gate	Requirement
Pilot pass	Tests 1-7 pass for the pilot group; controls fire as designed with no business-blocking false positives

Gate	Requirement
Expand pass	Tests 1-4 and 8 pass after re-scoping to `CASB Broad – All users – Expand`; API policies promoted only after pilot pass

6. Troubleshooting Guide — CASB & Cloud App Control with Netskope

Use `NSCASB-Validation.ps1` / `NSCASB-Discovery.ps1` for readiness signals and `NSCASB-Report.ps1` for the cloud-app usage and risk picture, `NSCASB-Remediation.ps1` (default `-WhatIf`) for safe fixes. Netskope console specifics are deferred to the official Netskope documentation.

A. Controls not firing / app not seen by policy

Symptom	Cause	Remediation
Activity not controlled	Traffic not steered through Netskope	Confirm steering for the user; run `NSCASB-Validation.ps1`; ensure the pilot user is in scope
App visible but actions not inspected	SSL/TLS inspection not covering the category	Enable inspection for the in-scope category; confirm the decryption certificate is trusted
Policy seems ignored	Policy scoped to the wrong group	Confirm scope is `CASB Pilot – Pilot users – Pilot`; run `NSCASB-Validation.ps1`

B. Over-blocking / business impact

- Replace a hard block with restrict or coach on the activity controls — Inline – Configure profile.
- Narrow the policy to the specific risky activity (for example upload only) instead of the whole app.
- Roll the control back to monitor-only, then re-tune before re-enabling enforcement.

C. Discovery / risk register gaps

Symptom	Cause	Remediation
New shadow-IT app not governed	Register stale	Re-run discovery; add the app to the app risk register with a disposition
Sanctioned app has no policy	Disposition decided but policy not built	Run `NSCASB-Remediation.ps1` (flags sanctioned apps lacking a policy); build the Sanctioned apps policy

D. API CASB issues

Symptom	Cause	Remediation
At-rest findings missing	API connection unhealthy/expired	Re-authorize the API connections — Sanctioned apps – Configure per Netskope's connector flow
Legitimate sharing quarantined	API response too aggressive	Return the connection response to alert/monitor mode; re-tune before active response

E. Identity / scoping

• Users not matching policy: confirm the user resolves in Entra ID and is in the scope group; rebuild membership with `NSCASB-Set-ScopeTargeting.ps1`.

F. Diagnostics quick reference

```
# Inventory + readiness signals (read-only)
.\NSCASB-Discovery.ps1
# Confirm steering/inspection readiness and pilot scoping (read-only)
.\NSCASB-Validation.ps1
# Cloud-app usage and risk report (read-only)
.\NSCASB-Report.ps1
# Preview safe fixes (no changes)
.\NSCASB-Remediation.ps1 -WhatIf
```

G. When to escalate

If steering and inspection are confirmed, the user is in the correct scope group, and a control still will not fire, capture the `NSCASB-Validation.ps1` output and the relevant Netskope policy/event logs and open a case with Netskope support.

7. Best Practices — CASB & Cloud App Control with Netskope

• Discover before you decide. Build the app risk register — All cloud apps – Discovery stage first; you cannot make a sound sanctioned/unsanctioned call on apps you cannot see.

• Govern without blocking. Prefer restrict and coach over block — coach users toward the sanctioned alternative instead of slamming the door, which only drives workarounds.

• Reserve block for the highest-risk apps. Use a hard block only for apps whose risk score and disposition on the register genuinely warrant it.

• Pilot every control. Scope every new policy to `CASB Pilot – Pilot users – Pilot` first; expand to `CASB Broad – All users – Expand` only after the pilot pass gate.

• Start API policies in monitor mode. Watch the API connections — Sanctioned apps – Configure surface

findings before turning on automated response, so you don't quarantine legitimate sharing.

• Control activities, not just apps. The real leverage is the activity controls — Inline – Configure

profile (upload, share, download), plus data-movement controls for sensitive content — not blanket app blocking.

• Scope to identity. Apply controls by Entra group, not by IP, so policy follows the user across

devices and networks.

• Keep the register current. Re-run discovery each cycle; new shadow-IT apps appear constantly —

score and dispose of them on the same register.

• Keep a rollback ready. Before enabling any block, confirm the scope-out / monitor-only / disable

path so live traffic can be restored fast.

• Document dispositions. Record why each app is sanctioned, restricted, coached, or blocked — for

audits, repeatability, and consistent decisions as the catalog grows.

8. Deliverables Summary — CASB & Cloud App Control with Netskope

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	What CASB is, inline vs API, discovery and risk scoring, control levers, govern-without-blocking
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Network)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered steps: discover, decide, inline + API controls, pilot, validate, expand
5. Testing Guide	Validation procedures with pass/fail gates
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations

PowerShell scripts

Script	What it does
`NSCASB-Discovery.ps1`	Read-only. Enumerates Entra enterprise applications as a cloud-app inventory signal and confirms Graph + group readiness; Netskope REST inventory flagged/deferred
`NSCASB-Validation.ps1`	Read-only. Confirms steering/inspection readiness signals and that the named CASB policies are scoped to the pilot group
`NSCASB-Report.ps1`	Read-only. Produces the cloud-app usage and risk report (sanctioned vs shadow IT, risk score, coverage gaps)
`NSCASB-Remediation.ps1`	Safe remediation of policy gaps; defaults to `-WhatIf`, applies only with `-Execute`
`NSCASB-Set-ScopeTargeting.ps1`	Builds/maintains the CASB scope groups (pilot and broad) in Entra ID

Named artifacts (reused verbatim across the package)

- app risk register — All cloud apps – Discovery stage
- Sanctioned apps policy – In-scope apps – Configure
- Unsanctioned apps policy – Shadow IT – Configure
- activity controls — Inline – Configure profile
- API connections — Sanctioned apps – Configure
- `CASB Pilot – Pilot users – Pilot` and `CASB Broad – All users – Expand`

What the buyer receives

- This implementation guide as a branded Word document and PDF.
- All five PowerShell scripts in a single ZIP.
- A repeatable process following the model: Assess -> Configure -> Pilot -> Validate -> Expand.

Outcome

Every cloud app visible and governed — shadow IT discovered and risk-scored, with inline and API controls that protect data without blocking the business — proven on a pilot before you scale.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

NSCASB-Discovery.ps1

Purpose: Builds a starting cloud-app inventory signal by enumerating Entra ID enterprise applications (service principals) — a corroborating signal for the sanctioned-app list that Netskope discovery refines — and confirms Microsoft Graph connectivity and that scope groups exist. The Netskope REST cloud-app discovery pull is flagged and deferred (see the NETSKOPE block). HOW TO RUN powershell -ExecutionPolicy Bypass -File .\NSCASB-Discovery.ps1 Requires the Microsoft Graph PowerShell SDK and delegated scopes: Application.Read.All, Group.Read.All. EXPECTED RESULTS A list of Entra enterprise applications (cloud apps) and a confirmation that Graph + the CASB scope groups are ready. No changes are made (read-only). .NOTES Network / CASB — read-only. Netskope console internals deferred to the official Netskope docs. #> [CmdletBinding()] param([string[]] $ScopeGroup = @('CASB Pilot - Pilot users - Pilot','CASB Broad - All users - Expand'))

How to run: powershell -ExecutionPolicy Bypass -File .\NSCASB-Discovery.ps1 Requires the Microsoft Graph PowerShell SDK and delegated scopes: Application.Read.All, Group.Read.All. EXPECTED RESULTS A list of Entra enterprise applications (cloud apps) and a confirmation that Graph + the CASB scope groups are ready. No changes are made (read-only). .NOTES Network / CASB — read-only. Netskope console internals deferred to the official Netskope docs. #> [CmdletBinding()] param([string[]] $ScopeGroup = @('CASB Pilot - Pilot users - Pilot','CASB Broad - All users - Expand'))

NSCASB-Remediation.ps1

Purpose: Detects and (with -Execute) fixes the safe, identity-side gaps this package can act on: a missing CASB scope group in Entra ID. It REPORTS — but never auto-changes — Netskope policy objects (those changes are deferred to the Netskope console/REST per the official docs). By default it only previews the action it would take. HOW TO RUN Preview (no changes): powershell -ExecutionPolicy Bypass -File .\NSCASB-Remediation.ps1 -WhatIf Apply identity fixes: powershell -ExecutionPolicy Bypass -File .\NSCASB-Remediation.ps1 -Execute Requires the Microsoft Graph PowerShell SDK and Group.ReadWrite.All (for -Execute). EXPECTED RESULTS A list of detected gaps and, in -WhatIf mode, the exact change that would be made. With -Execute, a missing scope group is created. Netskope policy gaps are reported only. .NOTES Network / CASB. Identity changes are gated behind -Execute; Netskope changes deferred to vendor docs. #> [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([switch] $Execute, [switch] $WhatIf, [string[]] $ScopeGroup = @('CASB Pilot - Pilot users - Pilot','CASB Broad - All users - Expand'))

How to run: Preview (no changes): powershell -ExecutionPolicy Bypass -File .\NSCASB-Remediation.ps1 -WhatIf Apply identity fixes: powershell -ExecutionPolicy Bypass -File .\NSCASB-Remediation.ps1 -Execute Requires the Microsoft Graph PowerShell SDK and Group.ReadWrite.All (for -Execute). EXPECTED RESULTS A list of detected gaps and, in -WhatIf mode, the exact change that would be made. With -Execute, a missing scope group is created. Netskope policy gaps are reported only. .NOTES Network / CASB. Identity changes are gated behind -Execute; Netskope changes deferred to vendor docs. #> [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([switch] $Execute, [switch] $WhatIf, [string[]] $ScopeGroup = @('CASB Pilot - Pilot users - Pilot','CASB Broad - All users - Expand'))

NSCASB-Report.ps1

Purpose: Produces a cloud-app usage and risk report. It builds the inventory side from Entra ID enterprise applications (which cloud apps exist / are integrated) and the scope-group side from Entra groups, then writes a CSV you can fold into the app risk register. The authoritative per-app risk scores and live usage come from Netskope discovery and are pulled via the Netskope REST API — that pull is flagged and deferred to the Netskope docs. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\NSCASB-Report.ps1 -OutCsv .\casb-report.csv Requires the Microsoft Graph PowerShell SDK and: Application.Read.All, Group.Read.All. EXPECTED RESULTS Console summary + a CSV listing each cloud app (inventory signal) with placeholders for Netskope risk score / disposition to be reconciled. No changes are made (read-only). .NOTES Network / CASB — read-only. Netskope console internals deferred to the official Netskope docs. #> [CmdletBinding()] param([string] $OutCsv = ".\casb-report.csv")

How to run: powershell -ExecutionPolicy Bypass -File .\NSCASB-Report.ps1 -OutCsv .\casb-report.csv Requires the Microsoft Graph PowerShell SDK and: Application.Read.All, Group.Read.All. EXPECTED RESULTS Console summary + a CSV listing each cloud app (inventory signal) with placeholders for Netskope risk score / disposition to be reconciled. No changes are made (read-only). .NOTES Network / CASB — read-only. Netskope console internals deferred to the official Netskope docs. #> [CmdletBinding()] param([string] $OutCsv = ".\casb-report.csv")

NSCASB-Set-ScopeTargeting.ps1

Purpose: Creates (if missing) the CASB scope groups used to target policy — the pilot group and the broad group — and optionally adds members to the pilot group. These Entra security groups are what the Netskope CASB policies are scoped to (Assess -> ... -> Expand). Group-to-policy binding inside Netskope is performed in the Netskope console and is deferred to its docs. HOW TO RUN Create the groups (idempotent): powershell -ExecutionPolicy Bypass -File .\NSCASB-Set-ScopeTargeting.ps1 Add pilot members by UPN: .\NSCASB-Set-ScopeTargeting.ps1 -PilotMember user1@contoso.com,user2@contoso.com Requires the Microsoft Graph PowerShell SDK and Group.ReadWrite.All (+ User.Read.All to add members). EXPECTED RESULTS The pilot and broad scope groups exist; any supplied pilot members are added. Idempotent — re-running makes no duplicate groups and skips members already present. .NOTES Network / CASB. Creates/updates Entra groups. Netskope policy binding deferred to the vendor docs. #> [CmdletBinding(SupportsShouldProcess = $true)] param([string] $PilotGroup = 'CASB Pilot - Pilot users - Pilot', [string] $BroadGroup = 'CASB Broad - All users - Expand', [string[]] $PilotMember = @())

How to run: Create the groups (idempotent): powershell -ExecutionPolicy Bypass -File .\NSCASB-Set-ScopeTargeting.ps1 Add pilot members by UPN: .\NSCASB-Set-ScopeTargeting.ps1 -PilotMember user1@contoso.com,user2@contoso.com Requires the Microsoft Graph PowerShell SDK and Group.ReadWrite.All (+ User.Read.All to add members). EXPECTED RESULTS The pilot and broad scope groups exist; any supplied pilot members are added. Idempotent — re-running makes no duplicate groups and skips members already present. .NOTES Network / CASB. Creates/updates Entra groups. Netskope policy binding deferred to the vendor docs. #> [CmdletBinding(SupportsShouldProcess = $true)] param([string] $PilotGroup = 'CASB Pilot - Pilot users - Pilot', [string] $BroadGroup = 'CASB Broad - All users - Expand', [string[]] $PilotMember = @())

NSCASB-Validation.ps1

Purpose: Confirms the local steering/inspection readiness signals on the device (the Netskope client is present and a decryption certificate is trusted) and confirms the CASB scope groups exist in Entra ID so

policies can be scoped to the pilot before expansion. Validating the Netskope policy objects themselves is done via the Netskope REST API and is flagged/deferred. HOW TO RUN Local readiness signals (run on a steered endpoint, elevated): powershell -ExecutionPolicy Bypass -File .\NSCASB-Validation.ps1 Add -CheckGroups to also confirm scope groups in Entra (needs Graph SDK, Group.Read.All). EXPECTED RESULTS Steering client present (yes/no), a Netskope decryption certificate trusted (yes/no), and — with -CheckGroups — the pilot/broad scope groups present. No changes are made (read-only). .NOTES Network / CASB — read-only. Netskope console internals deferred to the official Netskope docs. #> [CmdletBinding()] param([switch] $CheckGroups, [string[]] $ScopeGroup = @('CASB Pilot - Pilot users - Pilot','CASB Broad - All users - Expand'))

How to run: Local readiness signals (run on a steered endpoint, elevated): powershell -ExecutionPolicy Bypass -File .\NSCASB-Validation.ps1 Add -CheckGroups to also confirm scope groups in Entra (needs Graph SDK, Group.Read.All). EXPECTED RESULTS Steering client present (yes/no), a Netskope decryption certificate trusted (yes/no), and — with -CheckGroups — the pilot/broad scope groups present. No changes are made (read-only). .NOTES Network / CASB — read-only. Netskope console internals deferred to the official Netskope docs. #> [CmdletBinding()] param([switch] $CheckGroups, [string[]] $ScopeGroup = @('CASB Pilot - Pilot users - Pilot','CASB Broad - All users - Expand'))